efrei
INGÉNIEURS - NUMÉRIQUE
PARIS PANTHÉON-ASSAS UNIVERSITÉ

Industrialisation et déploiement d'une pipeline Data temps réel

TP CI/CD avec Jenkins et SonarQube, puis TP Kubernetes sur Minikube
Mastère Data Engineering et Intelligence Artificielle

J'ai d'abord rendu une API de ventes, Kafka, Spark et PostgreSQL testables et contrôlés à chaque commit, puis je les ai déployés dans un cluster Kubernetes local.

Auteur	Adam BELOUCIF
Numéro étudiant	20220055
Module	DevOps, tests logiciels et CI/CD
Promotion	M2 Data Engineering et IA, 2026-2027
Date du rapport	22 octobre 2026

Sommaire

Contexte et livrables 3

Architecture de la plateforme 4

Stratégie de tests 5

- Tests unitaires 5
- Tests d'intégration 6
- Scénario de bout en bout 7

Pipeline Jenkins 7

Analyse SonarQube et Quality Gate 9

- Mesure avant et après 9

Problèmes rencontrés et corrections 11

Déploiement sur Kubernetes 12

- Adaptations par rapport à Docker Compose 13
- État du cluster et test du flux 14
- Mise à l'échelle et suppression d'un Pod 14

Bilan 16

Contexte et livrables

Les deux TP portent sur le même projet fil rouge, une plateforme d'analyse des ventes en temps réel fournie par l'enseignant. Une API FastAPI reçoit des commandes et publie un événement dans le topic Kafka sales.orders. Un job PySpark Structured Streaming lit ce topic, écarte les commandes invalides et écrit le résultat dans la table processed_orders de PostgreSQL.

Le premier TP demandait d'industrialiser cette chaîne avec trois niveaux de tests, une pipeline Jenkins et un Quality Gate SonarQube. Le second reprend le même dépôt et le déploie sur Minikube avec des manifestes Kubernetes écrits à la main. J'ai travaillé sur la branche dev d'un fork publié sur GitHub, et le travail est proposé en Pull Request vers main.

56	100 %	0	5
tests automatisés	couverture du code applicatif	bug, vulnérabilité ou duplication	Deployments Kubernetes

Élément	Emplacement
Dépôt	github.com/Adam-Blf/real-time-sales-devops-tp, branche dev
Pull Request	github.com/Adam-Blf/real-time-sales-devops-tp/pull/1
Tests	tests/unit, tests/integration, tests/e2e
CI	Jenkinsfile, docker/jenkins, tools/quality_gate.py
Qualité	sonar-project.properties, tools/sonar_setup.py
Kubernetes	dossier k8s/
Sorties brutes	docs/evidence (logs Jenkins, Quality Gate, kubectl)

Tableau 1. Où trouver chaque livrable

Architecture de la plateforme

Toute l'infrastructure démarre avec une seule commande, docker compose up, y compris Jenkins et SonarQube. Le schéma suivant résume le flux de données et la chaîne de contrôle qui l'entoure.

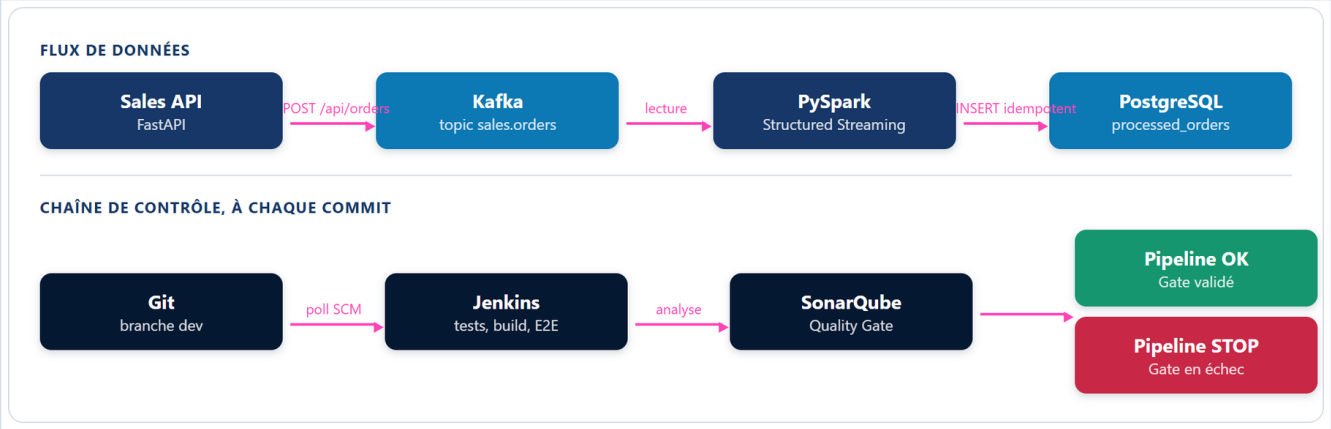


Figure 1. Flux de données et chaîne de contrôle

Avant d'écrire le moindre test, j'ai découpé l'API, qui tenait en un seul fichier. Les règles métier sont passées dans app/domain.py, sans aucune dépendance à Kafka. On y trouve le modèle Order, le catalogue, le calcul du montant et la construction de l'événement. La publication vit dans app/messaging.py avec un producteur Kafka unique, alors que le code d'origine en ouvrait un nouveau à chaque requête. Les routes ne font plus que relier les deux et traduire les erreurs en codes HTTP, 404 pour un produit inconnu, 422 pour une donnée invalide et 503 quand Kafka ne répond pas.

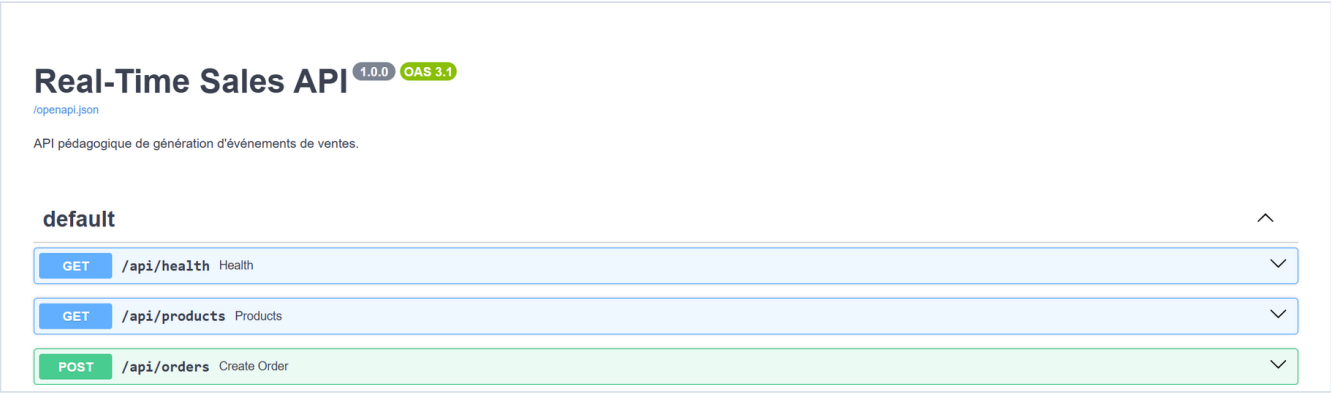


Figure 2. Les trois routes de l'API, documentation Swagger générée par FastAPI

Une fois la plateforme lancée, les dix conteneurs tournent côte à côte. Kafka et PostgreSQL déclarent une sonde de santé, sur laquelle s'appuient les dépendances de démarrage.

bash, plateforme Docker Compose

```
adam@efrei ~/real-time-sales-devops-tp $ docker compose ps
NAME                    IMAGE                                       STATUS
sales-api               real-time-sales-devops-tp-sales-api         Up 18 minutes
sales-jenkins           real-time-sales-devops-tp-jenkins           Up 21 minutes
sales-kafka             confluentinc/cp-kafka:7.6.14                Up 18 minutes (healthy)
sales-postgres          postgres:16-alpine                          Up 18 minutes (healthy)
sales-sonarqube         sonarqube:community                         Up 21 minutes
sales-sonarqube-db      postgres:16-alpine                          Up 21 minutes
sales-spark-master      apache/spark:3.5.7                          Up 18 minutes
sales-spark-streaming   real-time-sales-devops-tp-spark-streaming   Up 18 minutes
sales-spark-worker      apache/spark:3.5.7                          Up 18 minutes
sales-zookeeper         confluentinc/cp-zookeeper:7.6.14            Up 18 minutes
```

Figure 3. La plateforme démarrée avec Docker Compose

Stratégie de tests

J'ai suivi la pyramide classique, avec beaucoup de tests unitaires rapides, quelques tests d'intégration sur les vrais services et un seul scénario de bout en bout. Chaque niveau a sa propre étape Jenkins, et un échec dit tout de suite à quel étage se trouve le problème.

Niveau	Dossier	Ce qui est réel	Tests	Durée
Unitaire	tests/unit	rien, Kafka est simulé	50	1,5 s
Intégration	tests/integration	API, Kafka, Spark, PostgreSQL	5	8 s
End-to-End	tests/e2e	toute la plateforme	1	1,5 s

Tableau 2. Les trois niveaux de tests

Le sujet interdisait les attentes arbitraires du type time.sleep(30). Les tests d'intégration et E2E passent tous par une fonction wait_for qui interroge la condition à intervalle régulier et lève une erreur explicite, avec la dernière exception rencontrée, dès que le délai maximal est dépassé. Un test qui échoue dit ce qu'il attendait, au lieu de rester bloqué trente secondes pour rien.

Les adresses des services viennent de variables d'environnement. Les mêmes tests tournent depuis mon poste, où Kafka répond sur localhost:9092, et depuis Jenkins, qui joint les services par leur nom dans le réseau Docker.

Tests unitaires

Les cas listés par le sujet sont tous couverts, la plupart avec des tests paramétrés qui balayent les bornes. Le tableau ci-dessous fait la correspondance.

Cas demandé	Ce que vérifient les tests
Montant total	cinq calculs dont 3 x 100 = 300 et 3 x 0,1 = 0,3, arrondi au centime
Quantité	1 et 1000 acceptés, 0, -1, -50, 1001 et une chaîne refusés
Prix	0,01 à 100 000 acceptés, 0, négatif, au-delà du plafond et texte refusés
Identifiants	client et produit vides, d'un seul caractère ou absents refusés
Identifiant de commande	format ORD- suivi de 10 hexadécimaux, 1000 tirages tous distincts
Événement	tous les champs présents, valeurs recopiées, horodatage ISO 8601 avec fuseau
Routes HTTP	commande publiée, 404 sans publication, 422 sur quatre charges, 503

Tableau 3. Correspondance entre le sujet et les tests unitaires

La couverture du paquet app atteint 100 %, mesurée par pytest-cov et publiée dans SonarQube sous forme de fichier coverage.xml.

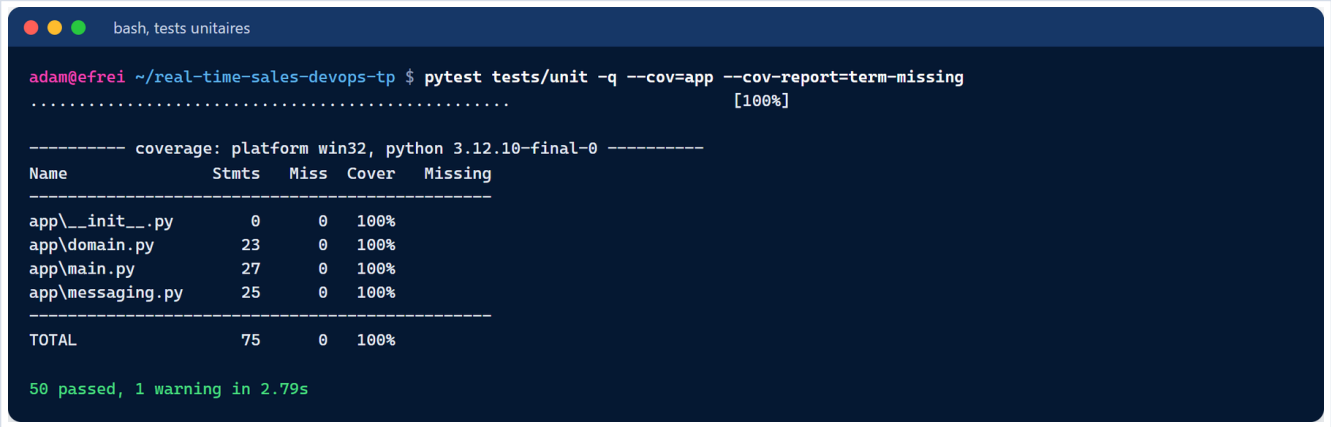


Figure 4. Les 50 tests unitaires et la couverture ligne par ligne

Tests d'intégration

Le test A vérifie le couple API et Kafka. Il poste une commande à l'API réellement démarrée, puis un consommateur ouvert dans un groupe neuf lit le topic jusqu'à trouver l'événement qui porte le même order_id. Le test compare ensuite le client, le produit, la quantité et le montant, ici 4 x 89,90 = 359,6.

Le test B écrit directement dans Kafka, sans passer par l'API, et attend la ligne correspondante dans processed_orders. Un second test publie une commande invalide suivie d'une commande valide qui sert de marqueur. Les deux tombent dans la même partition, et Spark les lit dans l'ordre. Quand le marqueur arrive en base, la commande invalide a forcément été traitée avant lui, et elle doit être absente. J'obtiens la preuve du filtrage sans attendre un délai choisi au hasard.

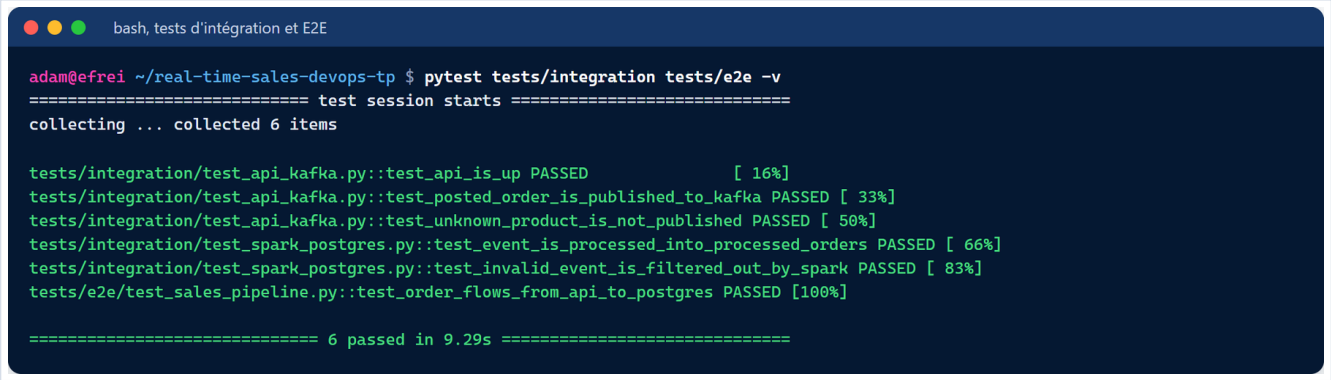


Figure 5. Les tests d'intégration et E2E, lancés contre la plateforme démarrée

Scénario de bout en bout

Le test E2E reprend la commande du sujet, le client C100 qui achète trois produits P001 à 100. Il vérifie que l'API répond, envoie la commande et attend la ligne en base. Le contrôle final porte sur le client, le produit, la quantité et le montant attendu de 300.

```
Given  l'infrastructure démarrée (GET /api/health renvoie UP)
When   POST /api/orders {customer_id: C100, product_id: P001, quantity: 3, unit_price: 100}
Then   une ligne apparaît dans processed_orders avec total_amount = 300
```

Figure 6. Le scénario E2E

Pipeline Jenkins

Je n'ai rien configuré à la main dans l'interface de Jenkins. Son image installe la CLI Docker, Python et les plugins nécessaires. Un fichier de configuration as code crée le compte administrateur, enregistre le jeton SonarQube comme identifiant et déclare le job real-time-sales-pipeline, branché sur la branche dev et interrogé toutes les deux minutes. Supprimer le conteneur et le recréer redonne exactement le même Jenkins.

```
Checkout -> Install -> Unit Tests -> Integration Tests -> Build -> Deploy
         -> E2E Tests -> SonarQube Analysis -> Quality Gate
```

Figure 7. Les étapes du Jenkinsfile

L'étape Build reconstruit les images de l'API et du job Spark, et l'étape Deploy les redémarre avant l'E2E. Le scénario de bout en bout valide le code du commit en cours, pas une image construite la veille. L'analyse SonarQube tourne dans un conteneur frère qui partage le workspace de Jenkins. L'étape Quality Gate lance ensuite un petit script Python qui attend la fin du traitement côté serveur, affiche chaque condition et renvoie un code d'erreur si le Gate n'est pas vert. Les rapports JUnit des trois niveaux et

le fichier de couverture sont archivés à chaque build.

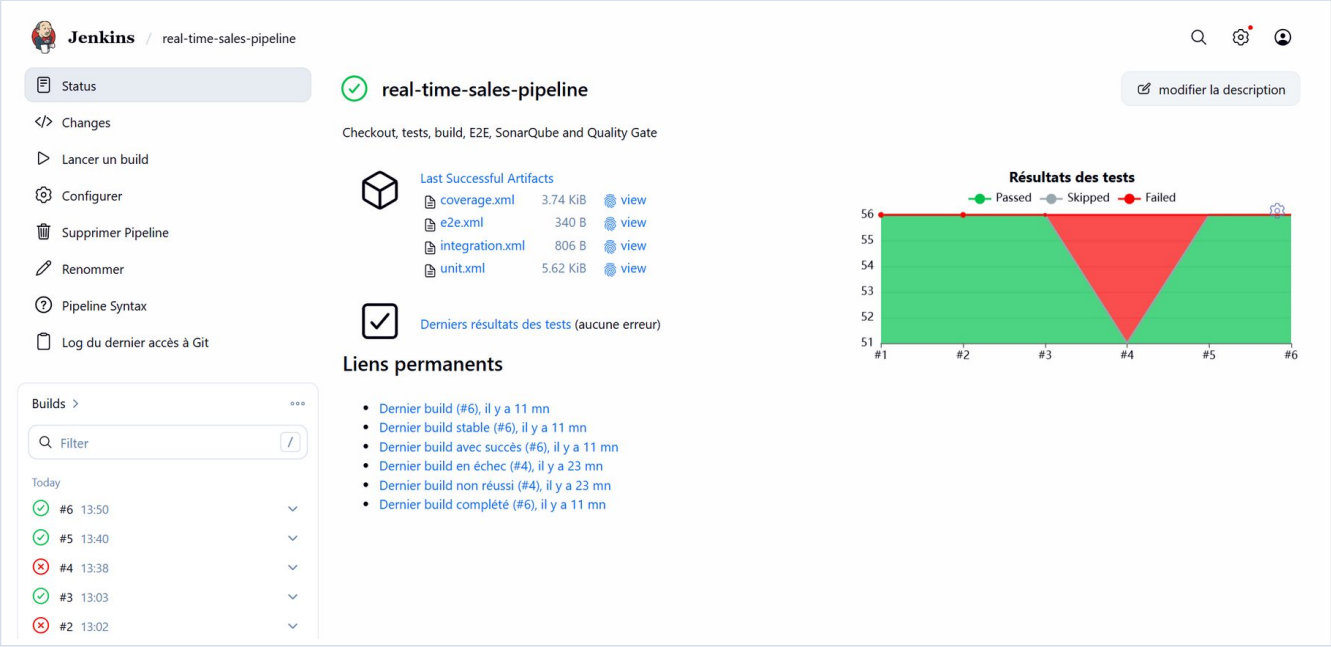


Figure 8. Historique du job. Les builds #2 et #4 se sont arrêtés, #3 et #5 sont passés

L'historique montre deux arrêts qui prouvent que la pipeline bloque vraiment. Le build #2 avait tous ses tests verts mais s'est arrêté à l'étape Quality Gate, faute de trouver le rapport du scanner (le correctif est décrit plus loin). Le build #4 s'est lancé tout seul après un push, à un moment où Kafka était arrêté sur mon poste. Il a échoué dès les tests d'intégration et n'est jamais allé jusqu'au build des images. Le build #5, lancé une fois la plateforme redémarrée, est vert sur le dernier commit.

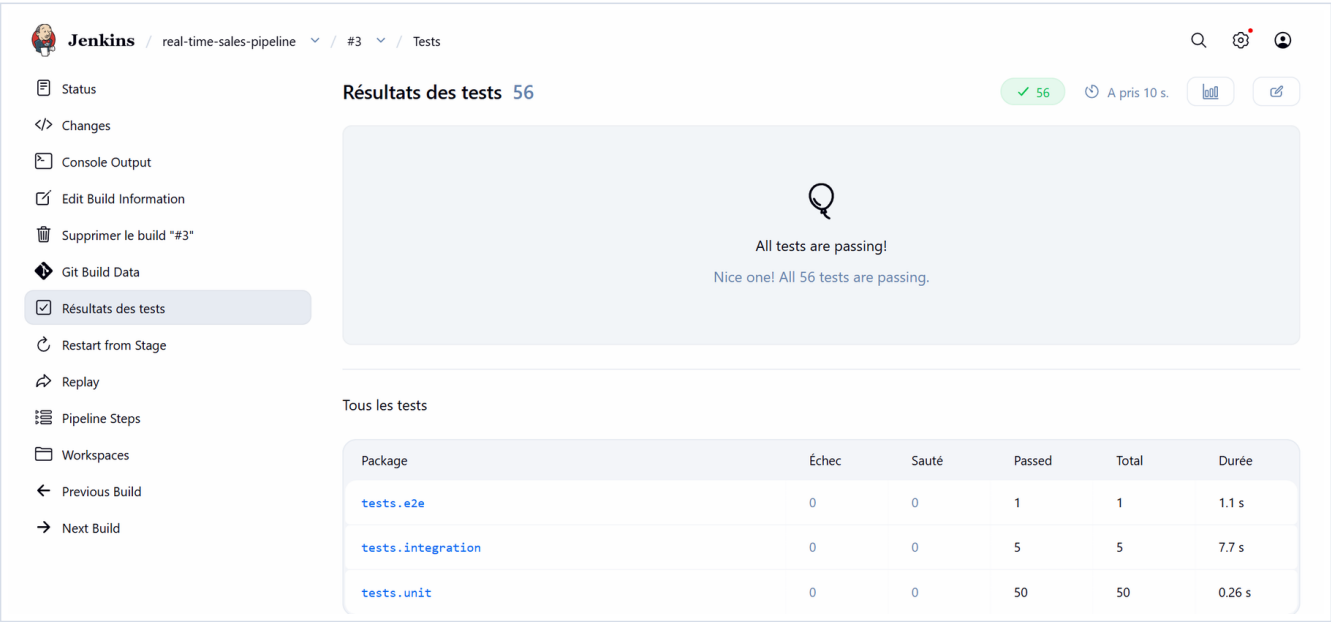


Figure 9. Résultats du build #3, 56 tests répartis sur les trois niveaux

La fin de la console du build #5 montre l'étape Quality Gate. Le script affiche chaque condition avec la valeur mesurée, et Jenkins conclut sur un succès.

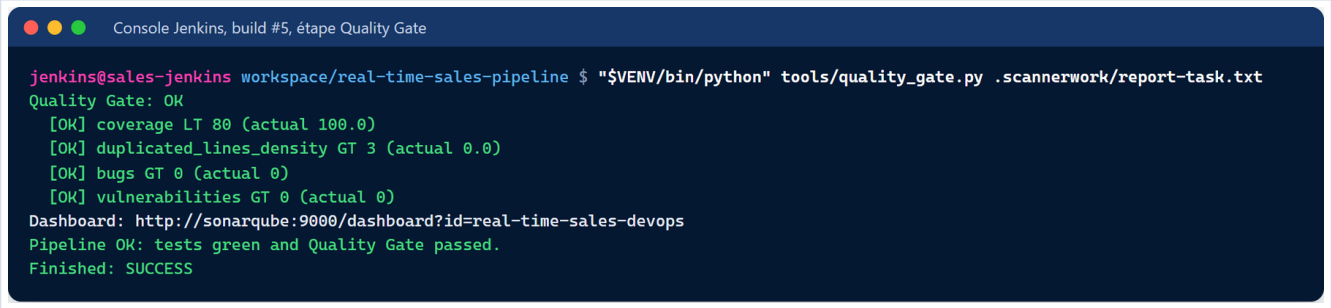


Figure 10. Sortie de l'étape Quality Gate dans la console Jenkins

Analyse SonarQube et Quality Gate

Le fichier sonar-project.properties analyse app, spark et tools, déclare tests comme code de test et lit le rapport de couverture. J'ai exclu le job Spark et l'outillage CI du calcul de couverture. Ils sont validés par les tests d'intégration et par Jenkins lui-même, et leur imposer des tests unitaires aurait surtout servi à gonfler un chiffre.

Le serveur se configure par script, avec tools/sonar_setup.py. Il crée le projet et le Quality Gate, puis écrit un jeton d'analyse dans le fichier .env, que Git ignore et que Docker Compose transmet à Jenkins.

Condition du Quality Gate	Seuil
Couverture	au moins 80 %
Bugs	aucun
Vulnérabilités	aucune
Duplications	au plus 3 %

Tableau 4. Conditions du Gate « Data Pipeline Gate »

Mesure avant et après

Pour comparer, j'ai analysé le code d'origine de l'enseignant, avec ses trois tests de départ, dans un projet SonarQube séparé et sous le même Gate. Il échoue sur deux conditions.

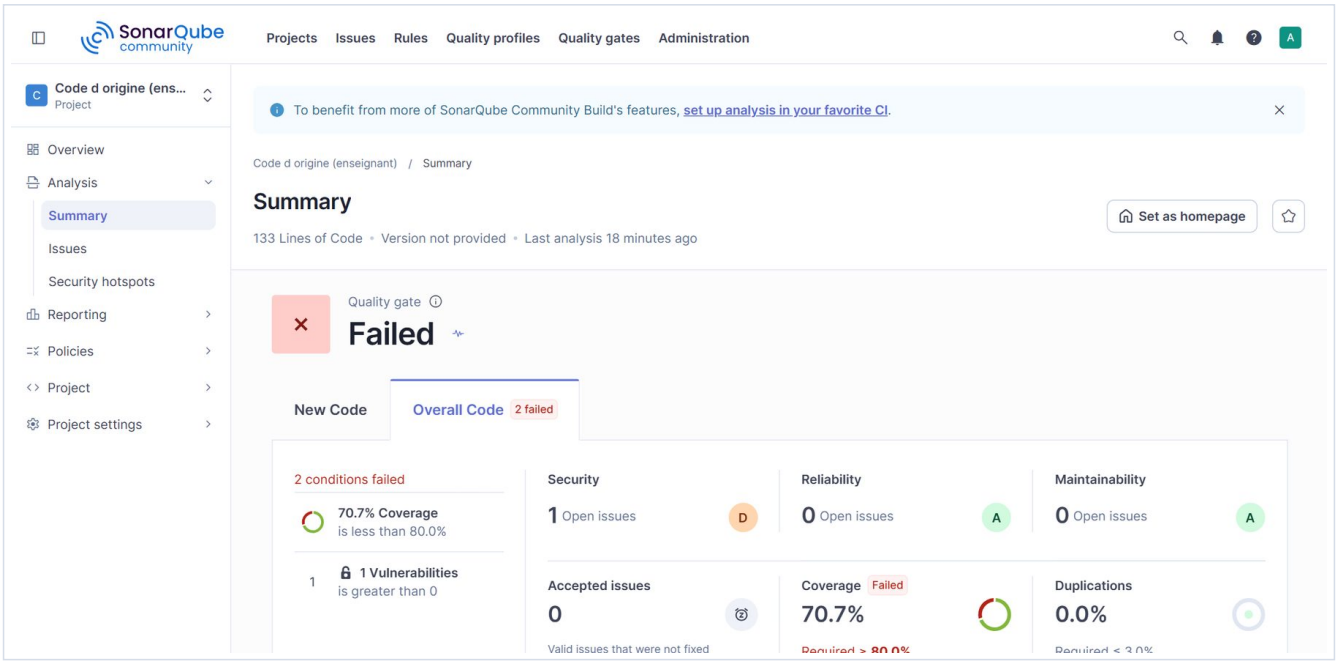


Figure 11. Code d'origine, Quality Gate en échec

La vulnérabilité, classée critique, vient du job Spark. Il écrivait son checkpoint dans /tmp, un répertoire où tout le monde peut écrire. La couverture de 70,7 % s'explique par les règles de validation, que les tests de départ ne vérifiaient qu'en partie. Il n'y avait en revanche ni bug, ni code smell, ni duplication.

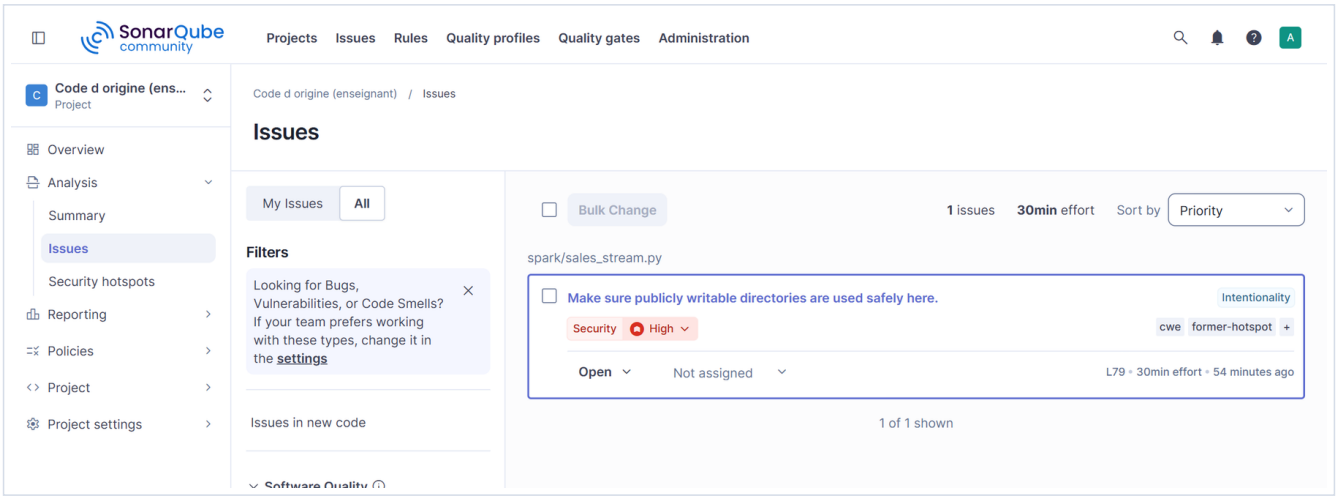


Figure 12. La vulnérabilité du code d'origine, ligne 79 de spark/sales_stream.py

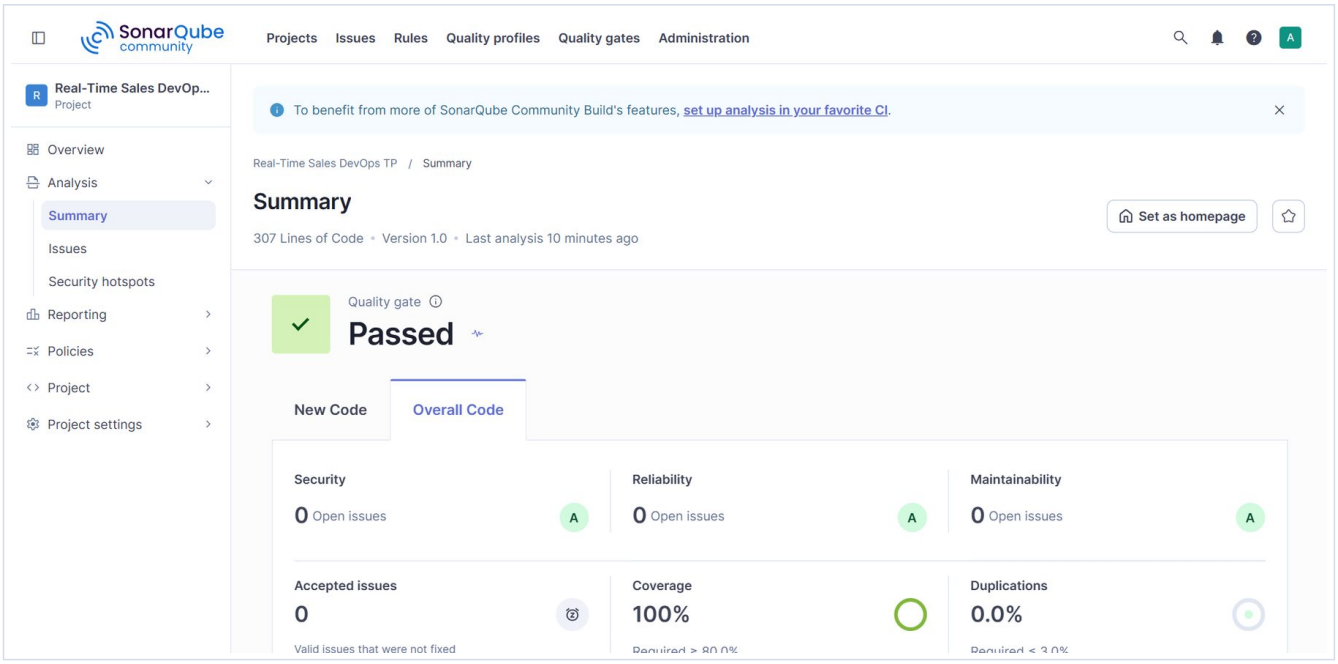


Figure 13. Code livré, Quality Gate validé

Mesure	Code d'origine	Code livré
Couverture	70,7 %	100 %
Bugs	0	0
Vulnérabilités	1 (critique)	0
Code smells	0	0
Duplications	0 %	0 %
Quality Gate	en échec	validé

Tableau 5. Comparaison des deux analyses

Problèmes rencontrés et corrections

Le dépôt de départ démarrait, mais plusieurs défauts n'apparaissaient qu'au premier test sérieux. Le plus gênant était le job Spark. Au premier démarrage, il s'abonnait au topic avant que Kafka ne l'ait créé, plantait sur UnknownTopicOrPartitionException et ne redémarrait jamais. J'ai ajouté un service kafka-init qui crée le topic, et Spark attend maintenant sa fin avant de démarrer.

Le redémarrage révélait le défaut suivant. Le checkpoint étant dans /tmp, il disparaissait avec le conteneur, et Spark relisait le topic depuis le début. Les commandes déjà en base provoquaient alors une violation de clé primaire. Le

checkpoint est désormais sur un volume, et l'insertion utilise ON CONFLICT DO NOTHING pour qu'un micro-batch rejoué ne casse plus rien. Ce même déplacement a fait disparaître la vulnérabilité relevée par SonarQube.

Symptôme	Cause	Correction
Spark meurt au premier démarrage	abonnement avant la création du topic	service kafka-init et redémarrage automatique
Doublons de clé primaire	checkpoint perdu, relecture depuis le début	volume de checkpoint et insertion idempotente
Écriture impossible depuis les exécuteurs	psycopg2 absent de l'image Spark	insertion côté driver, partition par partition
SonarQube refuse admin/admin	compte initial sans mot de passe utilisable	réinitialisation documentée, mot de passe aléatoire dans .env
Jenkins ignore sa configuration	fichier copié dans un dossier masqué par un volume	fichier déplacé hors du volume
Build #2 arrêté au Quality Gate	rapport du scanner écrit dans /tmp, perdu à la sortie	dossier de travail du scanner placé dans le workspace
Kafka ne repart pas après un arrêt brutal	nœud du broker encore présent dans ZooKeeper	redémarrage automatique du conteneur Kafka

Tableau 6. Défauts corrigés pendant le TP CI/CD

> **Secrets**
>
> Aucun mot de passe ni jeton n'est dans le dépôt. Les identifiants de Jenkins et de SonarQube sont générés au hasard et rangés dans .env, et Jenkins masque le jeton dans ses logs. GitGuardian a signalé deux jetons sur la Pull Request. Vérification faite, il s'agissait de deux identifiants de commit, qui ont la même forme que les anciens jetons SonarQube.

Déploiement sur Kubernetes

Le second TP reprend le même projet sur Minikube. Le cluster de mon poste hébergeait déjà les déploiements d'un autre cours dans le namespace default, et je n'ai pas voulu y toucher. Toute la plateforme vit dans un namespace dédié, sales.

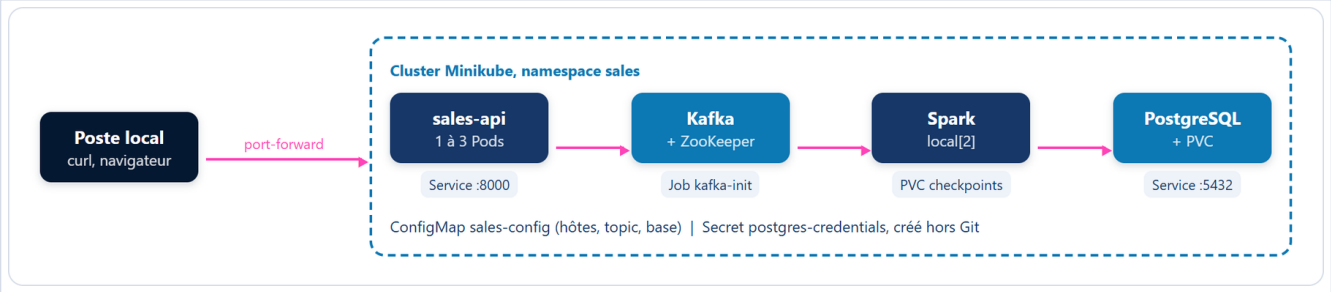


Figure 14. La plateforme déployée dans le cluster

Fichier	Ressources
00-namespace.yaml	Namespace sales
01-configmap.yaml	hôtes Kafka et PostgreSQL, topic, base, utilisateur
02-postgres-init.yaml	script de création de la table processed_orders
postgres.yaml	volume persistant, Deployment, Service postgres
kafka.yaml	ZooKeeper et Kafka (Deployments et Services), Job de création du topic
api-deployment.yaml	Deployment sales-api, sondes sur /api/health
api-service.yaml	Service sales-api sur le port 8000
spark-streaming.yaml	volume des checkpoints, Deployment spark-streaming

Tableau 7. Les manifestes du dossier k8s

Adaptations par rapport à Docker Compose

Les noms de services du compose ne valent rien dans le cluster. Les composants joignent les Services Kubernetes, kafka:9092 et postgres:5432, et ces adresses sont centralisées dans la ConfigMap. Kafka n'a plus besoin que d'un listener, puisque tous ses clients sont dans le cluster. Le mot de passe de PostgreSQL est dans un Secret que je crée à la main avec une valeur aléatoire, et aucun manifeste ne le contient.

Les images Confluent m'ont coûté le plus de temps. Pour chaque Service, Kubernetes injecte dans les Pods des variables comme KAFKA_PORT=tcp://10.110.182.112:9092. Or ces images lisent toute variable qui commence par KAFKA_ ou ZOOKEEPER_ comme un réglage, et refusent de démarrer. La ligne enableServiceLinks: false, posée sur chaque Pod, coupe cette injection. La sonde de disponibilité de Kafka, qui lançait la commande kafka-topics, posait un autre problème. Elle démarre une JVM complète et dépassait le délai de dix secondes sur un nœud Minikube chargé, si bien que Kafka redémarrait en boucle. Une sonde TCP sur le port 9092 a réglé le problème.

Spark tourne en mode local[2] dans un seul Deployment, avec la même image et le même job. Le couple master et worker du compose aurait doublé la mémoire

nécessaire pour un gain nul sur un nœud unique. Les images de l'API et de Spark sont construites dans Minikube et déclarées avec imagePullPolicy: Never.

État du cluster et test du flux

bash, cluster Minikube

```
adam@efrei ~/real-time-sales-devops-tp $ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   3d3h   v1.37.0

adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales get deployments,services
NAME                              READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/kafka             1/1     1            1           43m
deployment.apps/postgres          1/1     1            1           43m
deployment.apps/sales-api         1/1     1            1           43m
deployment.apps/spark-streaming   1/1     1            1           43m
deployment.apps/zookeeper         1/1     1            1           43m

NAME                 TYPE        CLUSTER-IP       EXTERNAL-IP   PORT(S)    AGE
service/kafka        ClusterIP   10.110.182.112   <none>        9092/TCP   43m
service/postgres     ClusterIP   10.104.132.42    <none>        5432/TCP   43m
service/sales-api    ClusterIP   10.104.170.184   <none>        8000/TCP   43m
service/zookeeper    ClusterIP   10.109.42.168    <none>        2181/TCP   43m
```

Figure 15. Le nœud Minikube et les ressources du namespace sales

J'ai exposé l'API avec kubectl port-forward, sur le port local 8001 parce que l'API du compose occupait déjà le 8000, puis envoyé la commande du sujet. L'API a renvoyé un montant de 300, et la ligne est apparue dans PostgreSQL quelques secondes plus tard. Le flux complet fonctionne dans le cluster.

bash, test du flux dans le cluster

```
adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales port-forward service/sales-api 8001:8000 &

adam@efrei ~/real-time-sales-devops-tp $ curl localhost:8001/api/health
{"status":"UP","service":"sales-api"}

adam@efrei ~/real-time-sales-devops-tp $ curl -X POST localhost:8001/api/orders -d '{"customer_id":"C100","product_id":"P001","quant
ity":3,"unit_price":100}'
{"customer_id":"C100","product_id":"P001","quantity":3,"unit_price":100.0,"order_id":"ORD-83F5C95BCD","total_amount":300.0,"timestam
p":"2026-10-22T13:58:08.057825+00:00"}

adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales exec deploy/postgres -- psql -U sales -d sales -c "select order_id, custom
er_id, product_id, quantity, total_amount from processed_orders where order_id='ORD-83F5C95BCD';"
    order_id    | customer_id | product_id | quantity | total_amount
----------------+-------------+------------+----------+--------------
 ORD-83F5C95BCD | C100        | P001       |        3 |       300.00
(1 row)
```

Figure 16. Une commande traverse Kafka et Spark jusqu'à PostgreSQL

Mise à l'échelle et suppression d'un Pod

Le passage à trois réplicas a créé deux Pods en quelques secondes, et le Deployment est passé à 3/3. Le Service sales-api répartit alors les requêtes entre les trois. Revenir à un réplica a supprimé les deux Pods en trop.

bash, mise à l'échelle

```
adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales scale deployment/sales-api --replicas=3
deployment.apps/sales-api scaled

adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales get pods -l app=sales-api -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
sales-api-b5db74958-d86m6    1/1     Running   0          6s    10.244.0.23   minikube   <none>           <none>
sales-api-b5db74958-jw7ph    1/1     Running   0          63s   10.244.0.22   minikube   <none>           <none>
sales-api-b5db74958-q4zcw    1/1     Running   0          6s    10.244.0.24   minikube   <none>           <none>

adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales get deployment sales-api
NAME        READY   UP-TO-DATE   AVAILABLE   AGE
sales-api   3/3     3            3           45m

adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales scale deployment/sales-api --replicas=1
deployment.apps/sales-api scaled
```

Figure 17. Passage de un à trois réplicas, puis retour à un

bash, suppression d'un Pod

```
adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales get pods -l app=sales-api
NAME                         READY   STATUS    RESTARTS   AGE
sales-api-b5db74958-g4j5d    1/1     Running   0          34m

adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales delete pod sales-api-b5db74958-g4j5d
pod "sales-api-b5db74958-g4j5d" deleted from sales namespace

adam@efrei ~/real-time-sales-devops-tp $ kubectl -n sales get pods -l app=sales-api -w --output-watch-events
EVENT      NAME                         READY   STATUS              RESTARTS   AGE
ADDED      sales-api-b5db74958-g4j5d    1/1     Running             0          34m
MODIFIED   sales-api-b5db74958-g4j5d    1/1     Terminating         0          34m
MODIFIED   sales-api-b5db74958-g4j5d    1/1     Terminating         0          34m
ADDED      sales-api-b5db74958-jw7ph    0/1     Pending             0          0s
MODIFIED   sales-api-b5db74958-jw7ph    0/1     Pending             0          0s
MODIFIED   sales-api-b5db74958-jw7ph    0/1     ContainerCreating   0          0s
MODIFIED   sales-api-b5db74958-jw7ph    0/1     ContainerCreating   0          1s
MODIFIED   sales-api-b5db74958-jw7ph    0/1     Running             0          1s
MODIFIED   sales-api-b5db74958-g4j5d    0/1     Completed           0          34m
MODIFIED   sales-api-b5db74958-g4j5d    0/1     Completed           0          34m
DELETED    sales-api-b5db74958-g4j5d    0/1     Completed           0          34m
MODIFIED   sales-api-b5db74958-jw7ph    1/1     Running             0          6s
MODIFIED   sales-api-b5db74958-jw7ph    1/1     Running             0          6s
```

Figure 18. Suppression d'un Pod de l'API et création de son remplaçant

Quand j'ai supprimé le Pod de l'API, le ReplicaSet a constaté qu'il restait zéro Pod pour un souhaité. Il en a créé un nouveau dans la même seconde, avant même que l'ancien ait fini de s'arrêter. Le remplaçant est devenu prêt six secondes plus tard, au moment où sa sonde /api/health a répondu. Kubernetes ne relance pas le Pod supprimé, il en crée un autre, avec un autre nom et une autre adresse IP. C'est pour cette raison qu'on s'adresse toujours au Service, dont l'adresse ne change pas.

Bilan

Dans le premier TP, les tests ont surtout servi à faire apparaître des défauts. Le test d'intégration Spark a révélé le plantage au démarrage et les doublons au redémarrage, et le Quality Gate a relevé le checkpoint dans /tmp que je n'avais pas vu.

Le passage à Kubernetes m'a surtout appris que la configuration d'un compose ne se recopie pas. Les noms DNS changent, les variables injectées par les Services entrent en conflit avec les images, et une sonde mal choisie suffit à faire tourner un service en boucle. La suite logique serait de brancher ce déploiement sur la pipeline Jenkins, que le sujet laissait hors du périmètre.